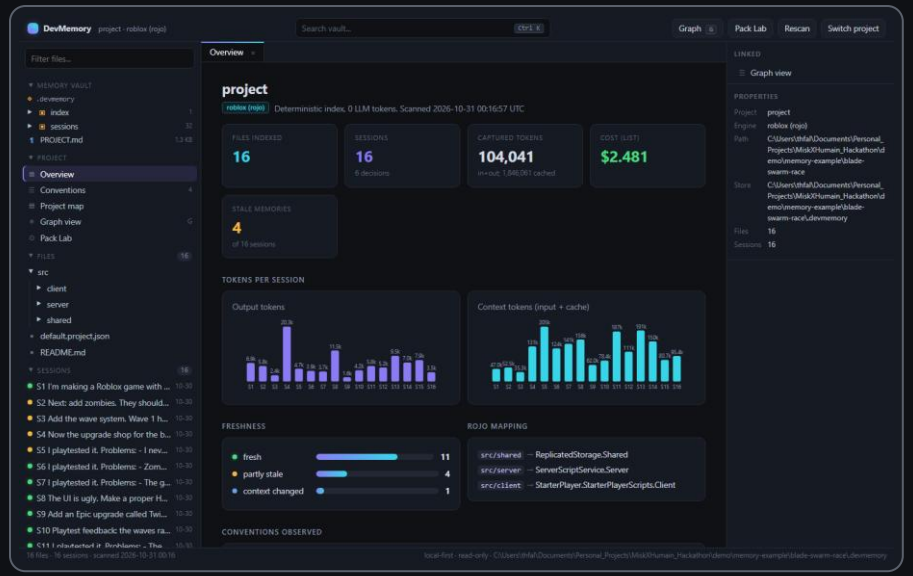
# DevMemory

## Memory that lets every new AI coding session start warm

A local-first memory layer for AI coding tools. It records every AI session as plain files, and before each new session it hands the model a small, sourced briefing of about 1,000 tokens: the user's earlier decisions in their own words, the files that matter, and warnings where the code has changed since. No AI is used to build the briefing.

**Thamer Alhussain**
HUMAIN–Misk Future AI Builders Hackathon · Challenge 2: Deliver AI with Less Compute




The DevMemory desktop app: a project's memory, browsable like a notes vault

# Every AI coding session starts cold

It re-reads the project, forgets earlier decisions, and guesses what the user meant

## 0 of 3

**restored the spec**

Codex runs without memory, asked to put back colors the user chose earlier

## 3 of 12

**history tasks solved**

without memory, on unseen tasks that need an earlier decision

## 11×

**cost of a bigger model**

per success for Sonnet without memory, and it still missed the accuracy bar

**Why this wastes compute.** Without memory, each session either re-explores the project (tokens spent rediscovering what was already known) or guesses (wrong results, more sessions to fix them). A bigger model or a longer context window does not fix this: the missing information is not in the code at all, it lives in earlier conversations. DevMemory keeps those conversations and sends the few lines that matter.

# How it works: scan, capture, pack, deliver

### 1 Scan

Builds a project map deterministically: files, symbols, the require graph, Roblox remotes and config keys, and the project's own conventions as observed in the code. 0 AI tokens; about 150 ms on Blade Swarm.

### 2 Capture

Records every AI session through documented interfaces: Claude Code stream-json, Codex JSON events, or a project snapshot diff for any other tool. Prompts, visible replies, tool calls, edits and file hashes, as plain files.

### 3 Pack

For a new request, ranks past sessions and files (BM25 plus dependency links) and assembles about 1,000 tokens: the user's earlier requirements verbatim, the session where the feature began, exact lines, and STALE flags.

### 4 Deliver

Injected automatically by a Claude Code hook, served to any MCP client by DevMemory's MCP server, or placed in any agent's prompt. The full archive never enters the prompt.

**Design choices:** plain files on the developer's machine · no vector database · no AI in capture or retrieval · every item carries its source and freshness · preserve everything, send only what earns its tokens

# Example: what a fresh session receives

Blade Swarm, a Roblox game built by prompting. The real pack for one request (excerpt)

```
## Conventions (observed in code)
- Tunables live in src/shared/Config.lua (51 keys)
- RemoteEvents only in src/shared/Remotes.lua
## What the user asked for earlier (verbatim)
- "Decisions: start with 2 blades, orbit radius 7 studs"
## Relevant past sessions
- [session dbf18bb2] User asked: "Now the upgrade
  shop ... +1 Blade (Rare, 60 coins), Sharper Blades
  (Common, 30) ... Greed +25% coins (Rare, 50) ..."
- [session 3e2bb0e2] User asked: "... rarity colors
  Common = gray, Rare = blue, Epic = purple ..."
  Outcome: "The colors live in a new Config.Ui
  table in src/shared/Config.lua"
## Relevant files
- src/shared/Config.lua [73 lines, used by 8]
- src/server/Shop.lua [164 lines]
```

### The request

A brand-new session is asked: "Add an Epic upgrade called Twin Spin... with the rarity colors we picked." The colors were chosen four sessions earlier.

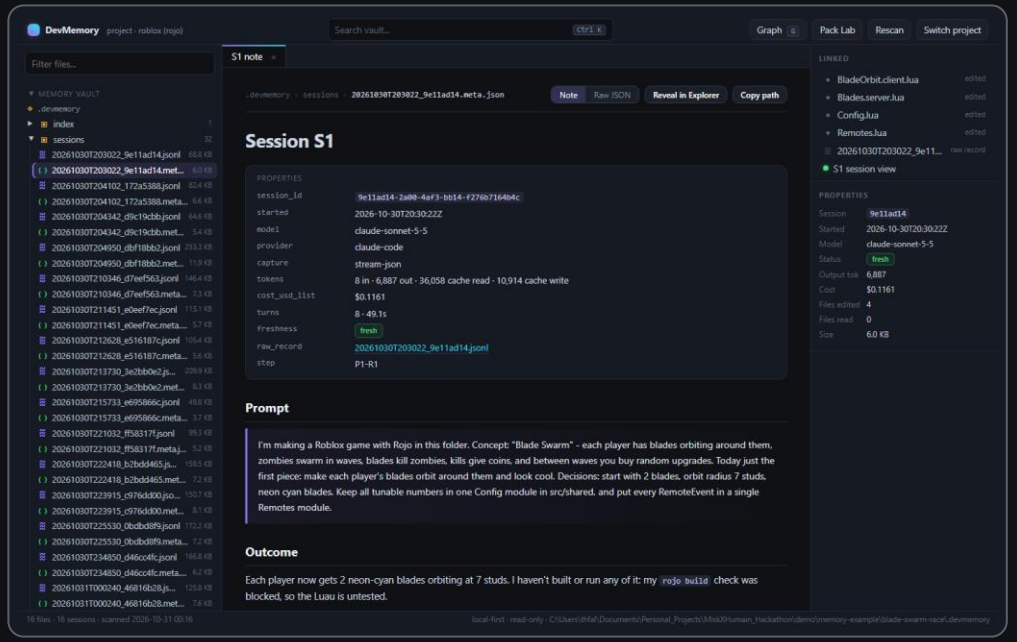
### What DevMemory found

Both earlier conversations, word for word: the shop design (rarities, prices) and the UI session where the user picked Common gray, Rare blue, Epic purple. About 1,150 tokens, built with 0 AI tokens.

### What the AI did

Claude Code read one file, added Twin Spin as an Epic upgrade in 13 seconds, and noted it "should show up purple like the colors we picked." (recorded live; shown in the video)

# What we built




A captured session in the app: the user's exact request, the outcome, files touched, freshness and cost

- **CLI:** devmem scan, run, pack, status, hook and app. Python standard library only; installs with pip.
- **Desktop app:** an Obsidian-style vault. Sessions as notes, raw records as timelines, a graph of files, sessions and decisions, and Pack Lab to preview exactly what a new session will receive.
- **Any AI tool:** an MCP server (get_context_pack, list and read sessions, begin and end session), a Codex adapter, and capture for any CLI or IDE agent. Tested with Claude Code and Codex.
- **Benchmark harness:** isolated headless runs, validated graders, Roblox Studio playtests through MCP, headless-Chrome web checks, reports with confidence intervals.

# How we tested

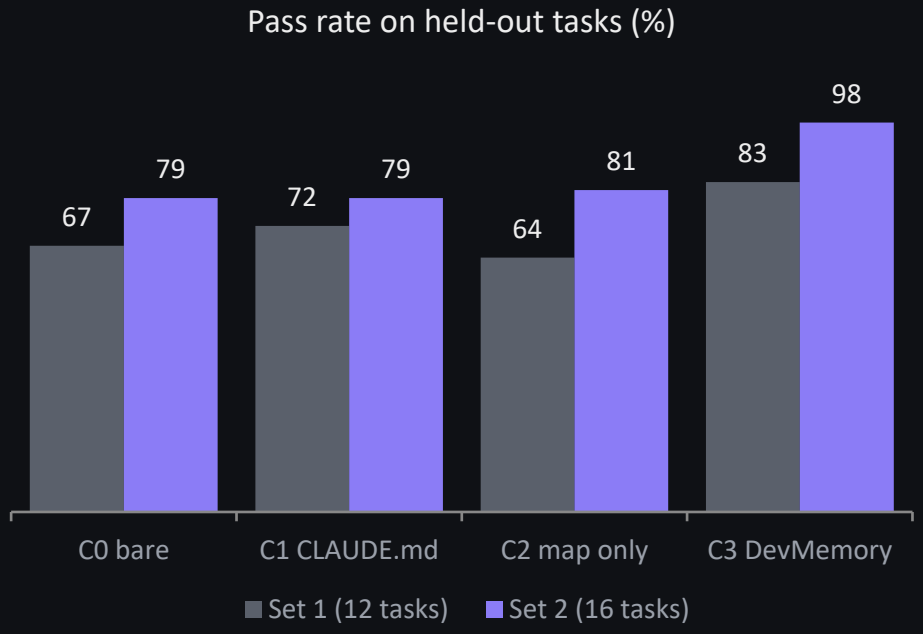
## A Fixed benchmark (unseen tasks)

- **Setups:** C0 bare session; C1 hand-written CLAUDE.md (the simple alternative); C2 DevMemory's map only; C3 full DevMemory. Same model (Claude Haiku 5.5), same isolated settings, 3 repeats.
- **Tasks:** 8 dev tasks for tuning. 28 held-out tasks in two sets, on a Roblox game and a Python app, written by a separate agent and hash-frozen before any run. Types: code, history, stale, unrelated.
- **Graders:** automatic and validated: the unmodified project must fail, a reference solution must pass.
- **Metric:** cost per successful task (all runs' cost / passes), counted only for setups above an 80% accuracy floor; 95% confidence intervals.

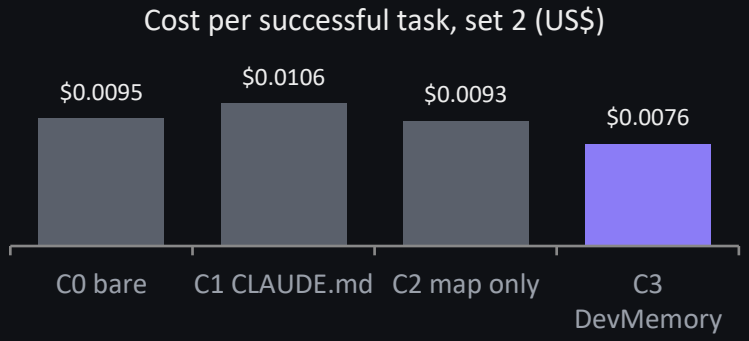
## B Real builds over many sessions

- **Roblox:** the Blade Swarm game built three ways (no memory, a self-maintained AGENTS.md, DevMemory), 16 fresh sessions each, automated Studio playtest after every prompt, human playtest at the end.
- **Next prompt:** chosen by a fixed rule: a bug report from failed checks, otherwise the next feature.
- **Web app:** a team task board raced the same way (8 features, API and headless-Chrome checks), two rounds.
- **Across tools:** memory recorded in Claude Code handed to Codex; isolated runs, 3 per variant.

# Unseen tasks: only DevMemory clears 80%

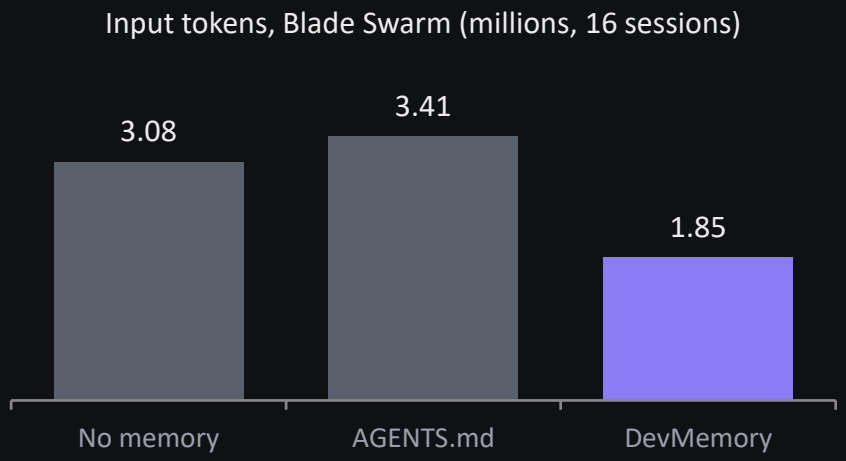



Accuracy floor 80%. Set 1 shown with the shipped code (DevMemory scored 86% when the set was frozen).




- **Accuracy:** DevMemory 83% and 98%; the others 64–81%.
- **Cost:** lowest per success on set 2, $0.0076 vs $0.0093–0.0106.
- **Where it comes from:** history tasks, 12/12 vs 3–4/12. Code, stale and unrelated tasks were similar for all.
- **Bigger model:** Sonnet without memory, $0.092 per success (11× more) at 75%.

# Building a real game: about half the tokens




| | None | AGENTS.md | DevMemory |
|---|---|---|---|
| Cost | $3.41 | $2.88 | **$2.48** |
| Playable (human) | yes | yes | **yes** |

Same prompts for all three; one run per setup (a case study, not a statistic).




- **What happened:** each feature was built in a fresh session. DevMemory used 1.85M input tokens vs 3.08M (no memory) and 3.41M (an AGENTS.md the agent kept itself), at the lowest cost. All three final games are playable.
- **Scripted 8-prompt build:** DevMemory 0.61M tokens vs 1.06M / 1.20M; the AGENTS.md build crashed on start in Studio.
- **Caveat:** the automated playtest was flaky and sent fix prompts for phantom bugs to every setup, so read the gap as indicative.

# What failed, and what we changed

## Web-app race, round 1: DevMemory lost

It built 3 of 8 features while the others built 8 of 8. A bug report never repeated the label colors, and the pack did not carry the session where the user had specified them. Fix: origin lineage (always include the session where a feature began) and verbatim requirement memory (keep the user's concrete values: colors, numbers, API paths).

**Re-test:** Round 2: all 8/8, no fixes. DevMemory cheapest: $0.74, 580k tok (vs 633k / 847k)

**Also:** on the set-1 Python project the hand-written file matched DevMemory (20 vs 19 of 24); on set 2 DevMemory led there (23 vs 17). The retrieval fix cost 3 points on set 1 (86% to 83%).




Scripted 8-prompt race: the AGENTS.md build passed every static check, yet crashed on start (WAVE 0, no blades). Code checks alone are not enough.

## Limitations (all disclosed in RESULTS.md)

- Benchmark projects are synthetic and small
- One run per setup in the multi-session builds
- The Roblox race's automated playtest was flaky (checked by a human at the end)
- Pack sizes are estimates (characters / 4); costs are API list-price equivalents

# Less compute, same or better results

### Cheaper model + memory

beat a bigger model without it: 83% vs 75% on the same tasks, at about a tenth of the cost per success

### ~Half the tokens

over a real multi-session game build, with all three results playable

### Works across tools

Claude Code memory: Codex right 3/3 vs 0/3, 56% fewer tokens

### The user's own words

are what make memory useful: requirements kept verbatim, never paraphrased

## Next

- **Install:** a one-command Claude Code plugin.
- **More tools:** test the MCP server in Cursor and VS Code.
- **Game dev:** a Roblox Studio playtest gate in the loop, then more engines.
- **Scale:** larger, real (non-synthetic) projects and more repeats.




github.com/Thamer-A/devmemory-hackathon · RESULTS.md (all numbers and disclosures) · bench/METHODOLOGY.md